

```
from google.colab import files

uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Online Retail.xlsx to Online Retail.xlsx

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import pandas as pd

df = pd.read_excel("Online Retail.xlsx")

print(df.shape)
df.head()
```

(541909, 8)

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	2010-12-01 08:26:00	2.55
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39

```
df = df.drop_duplicates()

df = df.dropna(subset=["Description"])

df["InvoiceNo"] = df["InvoiceNo"].astype(str)

df = df[
    ~df["InvoiceNo"].str.startswith("C")
]
```

```

df = df[df["Quantity"] > 0]

df = df[df["UnitPrice"] > 0]

df["Revenue"] = df["Quantity"] * df["UnitPrice"]

df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

df["Year"] = df["InvoiceDate"].dt.year
df["Month"] = df["InvoiceDate"].dt.month
df["YearMonth"] = df["InvoiceDate"].dt.to_period("M").astype(str)
df["DayOfWeek"] = df["InvoiceDate"].dt.day_name()
df["Hour"] = df["InvoiceDate"].dt.hour

print(df.shape)

```

(524878, 14)

```

total_revenue = df["Revenue"].sum()
total_orders = df["InvoiceNo"].nunique()
total_products = df["StockCode"].nunique()
total_customers = df["CustomerID"].nunique()
total_units = df["Quantity"].sum()

average_order_value = total_revenue / total_orders

print(f"Total Revenue: £{total_revenue:,.2f}")
print(f"Total Orders: {total_orders:,}")
print(f"Total Products: {total_products:,}")
print(f"Total Customers: {total_customers:,}")
print(f"Total Units Sold: {total_units:,}")
print(f"Average Order Value: £{average_order_value:,.2f}")

```

Total Revenue: £10,642,110.80
 Total Orders: 19,960
 Total Products: 3,922
 Total Customers: 4,338
 Total Units Sold: 5,572,420
 Average Order Value: £533.17

```

monthly_revenue = (
    df.groupby("YearMonth")["Revenue"]
      .sum()
      .reset_index()
)

monthly_revenue

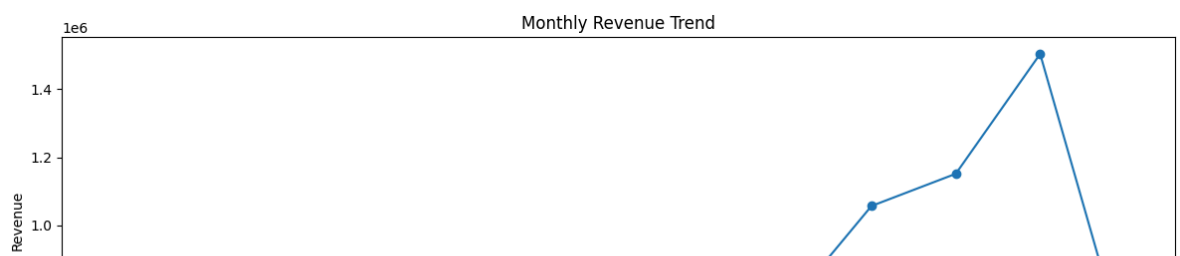
```

	YearMonth	Revenue
0	2010-12	821452.730
1	2011-01	689811.610
2	2011-02	522545.560
3	2011-03	716215.260
4	2011-04	536968.491
5	2011-05	769296.610
6	2011-06	760547.010
7	2011-07	718076.121
8	2011-08	757841.380
9	2011-09	1056435.192
10	2011-10	1151263.730
11	2011-11	1503866.780
12	2011-12	637790.330

```
plt.figure(figsize=(12, 5))

plt.plot(
    monthly_revenue["YearMonth"],
    monthly_revenue["Revenue"],
    marker="o"
)

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
monthly_revenue.loc[
    monthly_revenue["Revenue"].idxmax()
]
```

11

YearMonth	2011-11
------------------	---------

Revenue	1503866.78
----------------	------------

dtype: object

```
monthly_revenue.loc[
    monthly_revenue["Revenue"].idxmin()
]
```

2

YearMonth	2011-02
------------------	---------

Revenue	522545.56
----------------	-----------

dtype: object

```
top_products = (
    df.groupby("Description")["Revenue"]
      .sum()
      .sort_values(ascending=False)
      .head(10)
)

top_products
```

	Revenue
Description	
DOTCOM POSTAGE	206248.77
REGENCY CAKESTAND 3 TIER	174156.54
PAPER CRAFT , LITTLE BIRDIE	168469.60
WHITE HANGING HEART T-LIGHT HOLDER	106236.72
PARTY BUNTING	99445.23
JUMBO BAG RED RETROSPOT	94159.81
MEDIUM CERAMIC TOP STORAGE JAR	81700.92
POSTAGE	78101.88
Manual	77752.82
RABBIT NIGHT LIGHT	66870.03

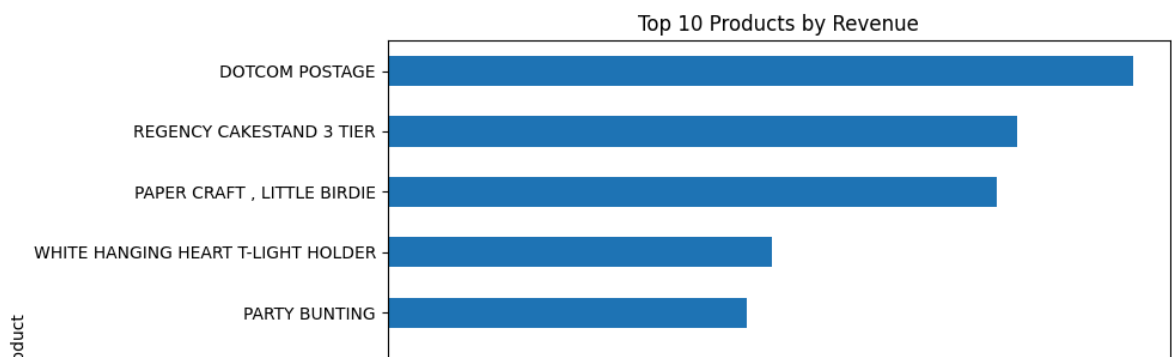
dtype: float64

```
plt.figure(figsize=(10, 6))

top_products.sort_values().plot(kind="barh")

plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product")

plt.tight_layout()
plt.show()
```



```
country_revenue = (
    df.groupby("Country")["Revenue"]
      .sum()
      .sort_values(ascending=False)
)

country_revenue.head(10)
```

	Revenue
Country	
United Kingdom	9001744.094
Netherlands	285446.340
EIRE	283140.520
Germany	228678.400
France	209625.370
Australia	138453.810
Spain	61558.560
Switzerland	57067.600
Belgium	41196.340
Sweden	38367.830

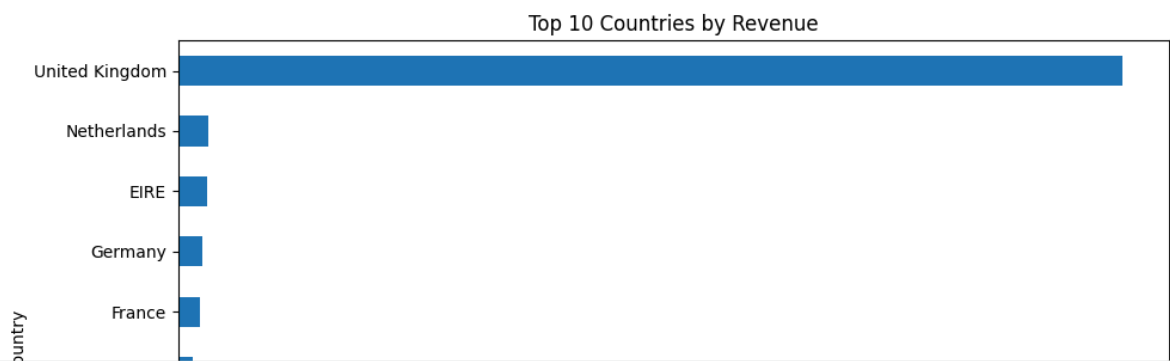
dtype: float64

```
plt.figure(figsize=(10, 6))

country_revenue.head(10).sort_values().plot(kind="barh")

plt.title("Top 10 Countries by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Country")

plt.tight_layout()
plt.show()
```



```
customer_revenue = (
    df.dropna(subset=["CustomerID"])
    .groupby("CustomerID")["Revenue"]
    .sum()
    .sort_values(ascending=False)
)

customer_revenue.head(10)
```

Revenue	
CustomerID	
14646.0	280206.02
18102.0	259657.30
17450.0	194390.79
16446.0	168472.50
14911.0	143711.17
12415.0	124914.53
14156.0	117210.08
17511.0	91062.38
16029.0	80850.84
12346.0	77183.60

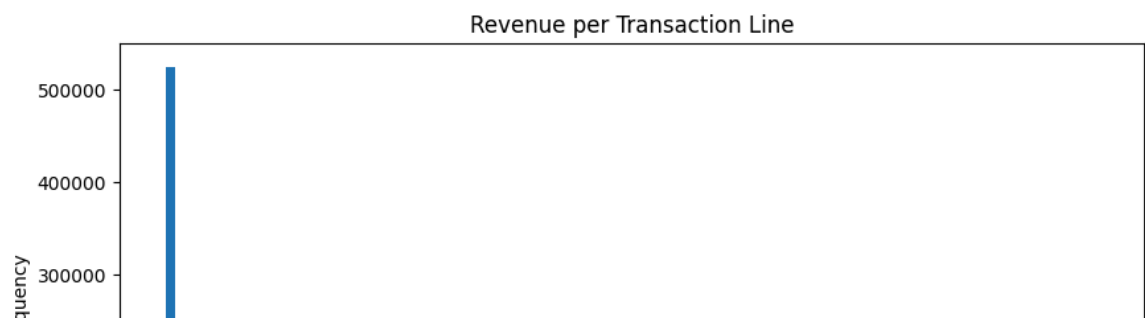
dtype: float64

```
plt.figure(figsize=(10, 5))

plt.hist(df["Revenue"], bins=100)

plt.title("Revenue per Transaction Line")
plt.xlabel("Revenue")
plt.ylabel("Frequency")

plt.show()
```



```
df.nlargest(
    20,
    "Revenue"
)[[
    "InvoiceNo",
    "StockCode",
    "Description",
    "Quantity",
    "UnitPrice",
    "Revenue",
    "Country"
]]
```



```

        .sum()
    )

    day_revenue

```

	Revenue					
268028	560373	M	Manual	1	4287.63	4287.6
DayOfWeek						
Friday	1837470.491	48185	DOORMAT FAIRY CAKE	670	6.35	4254.5
Monday	1775782.071		ASSORTED			
Sunday	806790.781	84879	COLOUR BIRD ORNAMENT	2880	1.45	4176.0
Thursday	2199292.570					
422351	573077	M	Manual	1	4161.06	4161.0
Tuesday	2175700.511					
Wednesday	1847074.380					

dtype: float64

```

day_order = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"
]

day_revenue = day_revenue.reindex(day_order)

day_revenue

```

	Revenue
DayOfWeek	
Monday	1775782.071
Tuesday	2175700.511
Wednesday	1847074.380
Thursday	2199292.570
Friday	1837470.491
Saturday	NaN
Sunday	806790.781

dtype: float64

```

# 1. Highest revenue month
monthly_revenue = (
    df.groupby("YearMonth")["Revenue"]
    .sum()

```

```

        .sort_values(ascending=False)
    )

print("Top 3 Revenue Months:")
print(monthly_revenue.head(3))

# 2. Top 3 products by revenue
top_products = (
    df.groupby("Description")["Revenue"]
      .sum()
      .sort_values(ascending=False)
)

print("\nTop 3 Products:")
print(top_products.head(3))

# 3. Top 3 countries by revenue
country_revenue = (
    df.groupby("Country")["Revenue"]
      .sum()
      .sort_values(ascending=False)
)

print("\nTop 3 Countries:")
print(country_revenue.head(3))

# 4. Largest transaction line
largest_transaction = df.nlargest(1, "Revenue")

print("\nLargest Transaction:")
print(
    largest_transaction[
        [
            "InvoiceNo",
            "StockCode",
            "Description",
            "Quantity",
            "UnitPrice",
            "Revenue",
            "Country"
        ]
    ].to_string(index=False)
)

# 5. Highest revenue day of week
day_order = [
    "Monday", "Tuesday", "Wednesday",
    "Thursday", "Friday", "Saturday", "Sunday"
]

day_revenue = (
    df.groupby("DayOfWeek")["Revenue"]
      .sum()
      .reindex(day_order)
)

print("\nRevenue by Day:")
print(day_revenue)

```

```
print("\nHighest Revenue Day:")
print(day_revenue.idxmax(), "-> £", f"{day_revenue.max():,.2f}")
```

Top 3 Revenue Months:

YearMonth

2011-11 1503866.780

2011-10 1151263.730

2011-09 1056435.192

Name: Revenue, dtype: float64

Top 3 Products:

Description

DOTCOM POSTAGE 206248.77

REGENCY CAKESTAND 3 TIER 174156.54

PAPER CRAFT , LITTLE BIRDIE 168469.60

Name: Revenue, dtype: float64

Top 3 Countries:

Country

United Kingdom 9001744.094

Netherlands 285446.340

EIRE 283140.520

Name: Revenue, dtype: float64

Largest Transaction:

InvoiceNo	StockCode	Description	Quantity	UnitPrice	Revenue
581483	23843	PAPER CRAFT , LITTLE BIRDIE	80995	2.08	168469.60

Revenue by Day:

DayOfWeek

Monday 1775782.071

Tuesday 2175700.511

Wednesday 1847074.380

Thursday 2199292.570

Friday 1837470.491

Saturday NaN

Sunday 806790.781

Name: Revenue, dtype: float64

Highest Revenue Day:

Thursday -> £ 2,199,292.57

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(
    monthly_revenue.sort_index().index,
    monthly_revenue.sort_index().values,
    marker="o"
)
```

```
plt.title("Monthly Revenue Trend")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Revenue (£)")
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```



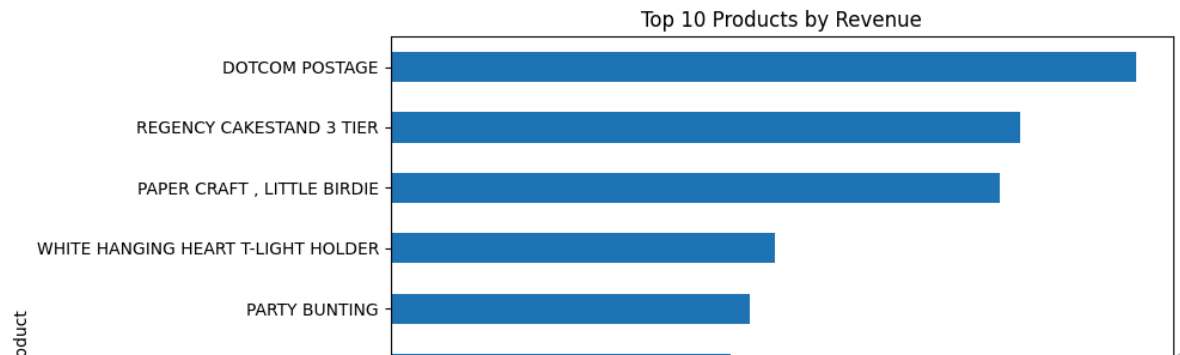
```
top_10_products = (
    df.groupby("Description")["Revenue"]
      .sum()
      .sort_values(ascending=False)
      .head(10)
)

plt.figure(figsize=(10, 6))

top_10_products.sort_values().plot(kind="barh")

plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue (£)")
plt.ylabel("Product")

plt.tight_layout()
plt.show()
```



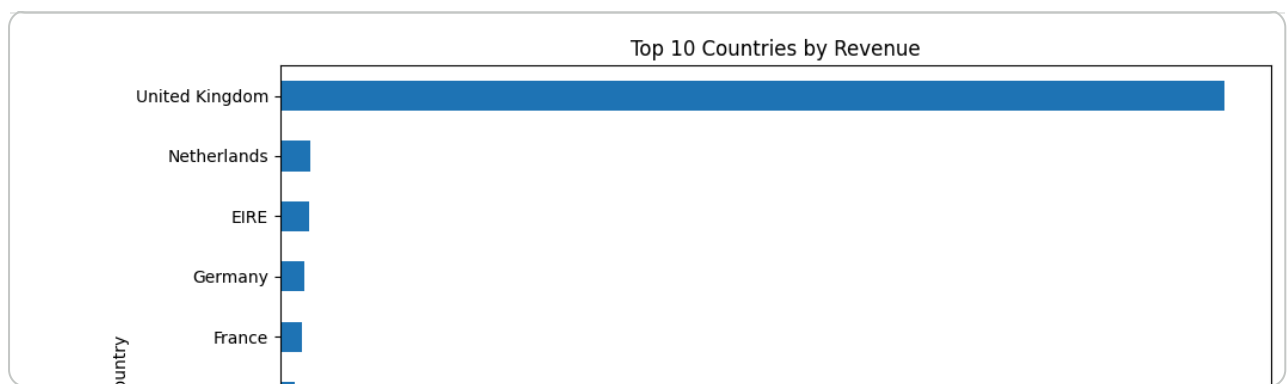
```
top_10_countries = (
    df.groupby("Country")["Revenue"]
      .sum()
      .sort_values(ascending=False)
      .head(10)
)

plt.figure(figsize=(10, 6))

top_10_countries.sort_values().plot(kind="barh")

plt.title("Top 10 Countries by Revenue")
plt.xlabel("Revenue (£)")
plt.ylabel("Country")

plt.tight_layout()
plt.show()
```



```
# Keep only transactions with a known CustomerID
customer_df = df.dropna(subset=["CustomerID"]).copy()

# Convert CustomerID to integer
customer_df["CustomerID"] = customer_df["CustomerID"].astype(int)

print("Customer transactions:", len(customer_df))
print("Unique customers:", customer_df["CustomerID"].nunique())
```

Customer transactions: 392692
Unique customers: 4338

```
reference_date = customer_df["InvoiceDate"].max() + pd.Timedelta(days=1)
print("Reference date:", reference_date)
```

Reference date: 2011-12-10 12:50:00

```
rfm = customer_df.groupby("CustomerID").agg(
    Recency=("InvoiceDate", lambda x:
            (reference_date - x.max()).days),

    Frequency=("InvoiceNo", "nunique"),

    Monetary=("Revenue", "sum")
).reset_index()

rfm.head()
```

	CustomerID	Recency	Frequency	Monetary
0	12346	326	1	77183.60
1	12347	2	7	4310.00
2	12348	75	4	1797.24
3	12349	19	1	1757.55
4	12350	310	1	334.40

```
rfm[["Recency", "Frequency", "Monetary"]].describe()
```

	Recency	Frequency	Monetary
count	4338.000000	4338.000000	4338.000000
mean	92.536422	4.272015	2048.688081
std	100.014169	7.697998	8985.230220
min	1.000000	1.000000	3.750000
25%	18.000000	1.000000	306.482500
50%	51.000000	2.000000	668.570000
75%	142.000000	5.000000	1660.597500
max	374.000000	209.000000	280206.020000

```
rfm["R_Score"] = pd.qcut(
    rfm["Recency"],
    5,
    labels=[5, 4, 3, 2, 1]
)

rfm["F_Score"] = pd.qcut(
    rfm["Frequency"].rank(method="first"),
    5,
    labels=[1, 2, 3, 4, 5]
)

rfm["M_Score"] = pd.qcut(
    rfm["Monetary"].rank(method="first"),
    5,
    labels=[1, 2, 3, 4, 5]
)
```

```
rfm["RFM_Score"] = (
    rfm["R_Score"].astype(int) +
    rfm["F_Score"].astype(int) +
    rfm["M_Score"].astype(int)
)
```

```
def segment_customer(row):
    r = int(row["R_Score"])
    f = int(row["F_Score"])
    m = int(row["M_Score"])

    if r >= 4 and f >= 4 and m >= 4:
        return "Champions"

    elif r >= 3 and f >= 4:
        return "Loyal Customers"

    elif r >= 4 and f <= 2:
        return "New Customers"

    elif r <= 2 and f >= 3 and m >= 3:
        return "At Risk"
```

```

        elif r <= 2 and f <= 2:
            return "Lost Customers"

        else:
            return "Potential Loyalists"

rfm["Segment"] = rfm.apply(segment_customer, axis=1)

```

```

segment_counts = (
    rfm["Segment"]
    .value_counts()
)

print(segment_counts)

```

```

Segment
Lost Customers      1065
Potential Loyalists 1041
Champions           957
Loyal Customers     503
At Risk             453
New Customers       319
Name: count, dtype: int64

```

```

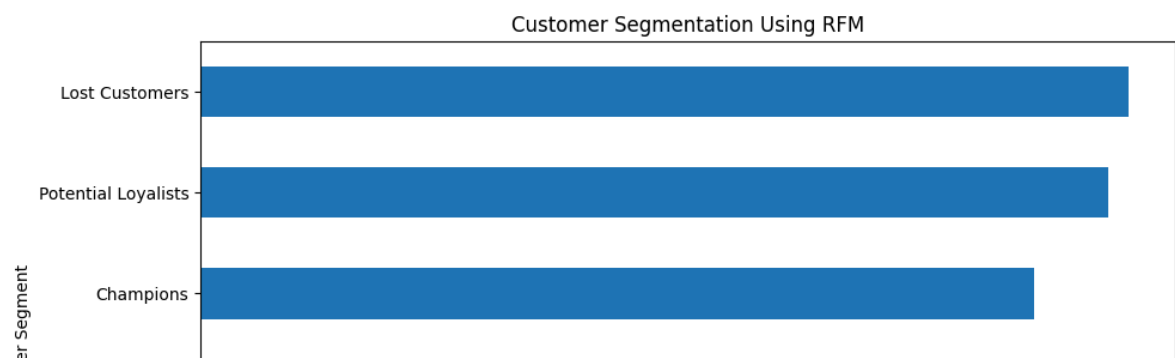
plt.figure(figsize=(10, 6))

segment_counts.sort_values().plot(kind="barh")

plt.title("Customer Segmentation Using RFM")
plt.xlabel("Number of Customers")
plt.ylabel("Customer Segment")

plt.tight_layout()
plt.show()

```



```

segment_analysis = (
    rfm.groupby("Segment")
    .agg(
        Customers=("CustomerID", "count"),
        Revenue=("Monetary", "sum"),
        Avg_Recency=("Recency", "mean"),
        Avg_Frequency=("Frequency", "mean"),
        Avg_Monetary=("Monetary", "mean")
    )
    .sort_values("Revenue", ascending=False)
)

segment_analysis

```

	Customers	Revenue	Avg_Recency	Avg_Frequency	Avg_Moneta
Segment					
Champions	957	5791640.740	12.824451	11.123302	6051.8712
Loyal Customers	503	895651.561	38.023857	5.077535	1780.6194
Potential Loyalists	1041	796897.150	65.230548	1.958694	765.5111
At Risk	453	739477.551	141.613687	3.807947	1632.4007
Lost Customers	1065	518322.152	217.897653	1.101408	486.6874
New Customers	319	145219.740	18.517241	1.241379	455.2342

```
champions = (
    rfm[rfm["Segment"] == "Champions"]
    .sort_values("Monetary", ascending=False)
)

champions.head(10)
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Scor
1689	14646	2	73	280206.02	5	5	
4201	18102	1	60	259657.30	5	5	
3728	17450	8	46	194390.79	5	5	
1879	14911	1	201	143711.17	5	5	
55	12415	24	21	124914.53	4	5	
1333	14156	10	55	117210.08	5	5	
3771	17511	3	31	91062.38	5	5	
3176	16684	4	28	66653.56	5	5	
1289	14096	4	17	65164.79	5	5	
996	13694	4	50	65039.62	5	5	

```
at_risk = (
    rfm[rfm["Segment"] == "At Risk"]
    .sort_values("Monetary", ascending=False)
)

at_risk.head(10)
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score
2502	15749	235	3	44534.30	1	3	
2011	15098	182	3	39916.50	1	3	
50	12409	79	3	11072.67	2	3	
2814	16180	100	8	10254.18	2	5	
566	13093	276	8	7832.47	1	5	
485	12980	158	9	7374.90	2	5	
3222	16745	87	17	7180.70	2	5	
519	13027	114	6	6912.00	2	5	
2637	15939	90	15	6115.01	2	5	
1293	14101	74	6	5976.79	2	5	

```
# =====
# PHASE 4 – RFM CUSTOMER SEGMENTATION
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Load the dataset
# -----

df = pd.read_excel("Online Retail.xlsx")

# -----
# 2. Clean the data
# -----

# Remove duplicate rows
df = df.drop_duplicates()

# Remove rows with missing product descriptions
df = df.dropna(subset=["Description"])

# Convert InvoiceNo to string
df["InvoiceNo"] = df["InvoiceNo"].astype(str)

# Remove cancelled invoices
df = df[~df["InvoiceNo"].str.startswith("C")]

# Keep only positive quantities
df = df[df["Quantity"] > 0]

# Keep only positive prices
df = df[df["UnitPrice"] > 0]

# -----
# 3. Create Revenue
# -----
```

```

df["Revenue"] = df["Quantity"] * df["UnitPrice"]

# Convert InvoiceDate
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

# -----
# 4. Keep customers with CustomerID
# -----

customer_df = df.dropna(subset=["CustomerID"]).copy()

# Convert CustomerID to integer
customer_df["CustomerID"] = (
    customer_df["CustomerID"].astype(int)
)

print("Customer transactions:", len(customer_df))
print("Unique customers:", customer_df["CustomerID"].nunique())

# -----
# 5. Define reference date
# -----

reference_date = (
    customer_df["InvoiceDate"].max()
    + pd.Timedelta(days=1)
)

print("\nReference date:", reference_date)

# -----
# 6. Calculate RFM
# -----

rfm = (
    customer_df
    .groupby("CustomerID")
    .agg(
        Recency=(
            "InvoiceDate",
            lambda x: (reference_date - x.max()).days
        ),
        Frequency=(
            "InvoiceNo",
            "nunique"
        ),
        Monetary=(
            "Revenue",
            "sum"
        )
    )
    .reset_index()
)

print("\nRFM Data:")
display(rfm.head())

# -----
# 7. RFM descriptive statistics
# -----

print("\nRFM Statistics:")
display(

```

```

    rfm[
        ["Recency", "Frequency", "Monetary"]
    ].describe()
)

# -----
# 8. Create RFM scores
# -----

# Recency:
# Recent customers = higher score

rfm["R_Score"] = pd.qcut(
    rfm["Recency"],
    q=5,
    labels=[5, 4, 3, 2, 1]
)

# Frequency:
# More orders = higher score

rfm["F_Score"] = pd.qcut(
    rfm["Frequency"].rank(method="first"),
    q=5,
    labels=[1, 2, 3, 4, 5]
)

# Monetary:
# Higher spending = higher score

rfm["M_Score"] = pd.qcut(
    rfm["Monetary"].rank(method="first"),
    q=5,
    labels=[1, 2, 3, 4, 5]
)

# -----
# 9. Calculate total RFM score
# -----

rfm["RFM_Score"] = (
    rfm["R_Score"].astype(int)
    + rfm["F_Score"].astype(int)
    + rfm["M_Score"].astype(int)
)

# -----
# 10. Customer segmentation
# -----

def segment_customer(row):

    r = int(row["R_Score"])
    f = int(row["F_Score"])
    m = int(row["M_Score"])

    # Recent + frequent + high spending
    if r >= 4 and f >= 4 and m >= 4:
        return "Champions"

    # Frequent recent customers
    elif r >= 3 and f >= 4:
        return "Loyal Customers"

```

```

# Recent but not frequent
elif r >= 4 and f <= 2:
    return "New Customers"

# Previously valuable but becoming inactive
elif r <= 2 and f >= 3 and m >= 3:
    return "At Risk"

# Low engagement
elif r <= 2 and f <= 2:
    return "Lost Customers"

# Everyone else
else:
    return "Potential Loyalists"

rfm["Segment"] = rfm.apply(
    segment_customer,
    axis=1
)

# -----
# 11. Customer count by segment
# -----

segment_counts = (
    rfm["Segment"]
    .value_counts()
)

print("\nCustomer Count by Segment:")
display(segment_counts)

# -----
# 12. Visualize customer segments
# -----

plt.figure(figsize=(10, 6))

segment_counts.sort_values().plot(
    kind="barh"
)

plt.title("Customer Segmentation Using RFM")
plt.xlabel("Number of Customers")
plt.ylabel("Customer Segment")

plt.tight_layout()
plt.show()

# -----
# 13. Revenue by customer segment
# -----

segment_analysis = (
    rfm
    .groupby("Segment")
    .agg(
        Customers=("CustomerID", "count"),
        Revenue=("Monetary", "sum"),
        Avg_Recency=("Recency", "mean"),
        Avg_Frequency=("Frequency", "mean"),
        Avg_Monetary=("Monetary", "mean")
    )

```

```

    )
    .sort_values(
        "Revenue",
        ascending=False
    )
)

print("\nSegment Analysis:")
display(segment_analysis)

# -----
# 14. Revenue by segment chart
# -----

plt.figure(figsize=(10, 6))

segment_analysis["Revenue"].sort_values().plot(
    kind="barh"
)

plt.title("Revenue Contribution by Customer Segment")
plt.xlabel("Revenue (£)")
plt.ylabel("Customer Segment")

plt.tight_layout()
plt.show()

# -----
# 15. Top Champions
# -----

champions = (
    rfm[
        rfm["Segment"] == "Champions"
    ]
    .sort_values(
        "Monetary",
        ascending=False
    )
)

print("\nTop 10 Champions:")
display(champions.head(10))

# -----
# 16. High-value At-Risk customers
# -----

at_risk = (
    rfm[
        rfm["Segment"] == "At Risk"
    ]
    .sort_values(
        "Monetary",
        ascending=False
    )
)

print("\nTop 10 At-Risk Customers:")
display(at_risk.head(10))

# -----
# 17. Save RFM results
# -----

```

```
rfm.to_csv(  
    "rfm_customer_segments.csv",  
    index=False  
)  
  
print("\nRFM analysis completed successfully!")
```

Customer transactions: 392692
Unique customers: 4338

Reference date: 2011-12-10 12:50:00

RFM Data:

	CustomerID	Recency	Frequency	Monetary
0	12346	326	1	77183.60
1	12347	2	7	4310.00
2	12348	75	4	1797.24
3	12349	19	1	1757.55
4	12350	310	1	334.40

RFM Statistics:

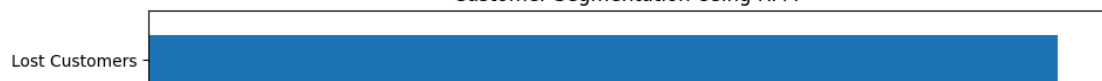
	Recency	Frequency	Monetary
count	4338.000000	4338.000000	4338.000000
mean	92.536422	4.272015	2048.688081
std	100.014169	7.697998	8985.230220
min	1.000000	1.000000	3.750000
25%	18.000000	1.000000	306.482500
50%	51.000000	2.000000	668.570000
75%	142.000000	5.000000	1660.597500
max	374.000000	209.000000	280206.020000

Customer Count by Segment:

	count
Segment	
Lost Customers	1065
Potential Loyalists	1041
Champions	957
Loyal Customers	503
At Risk	453
New Customers	319

dtype: int64

Customer Segmentation Using RFM



Correlation between RFM variables

```
rfm_correlation = rfm[
    ["Recency", "Frequency", "Monetary"]
].corr()
```

```
print("RFM Correlation Matrix:")
display(rfm_correlation)
```

RFM Correlation Matrix:

At Risk

Recency Frequency Monetary

Recency	1.000000	-0.260578	-0.121831
Frequency	-0.260578	1.000000	0.552780
Monetary	-0.121831	0.552780	1.000000

of Customers

Segment Analysis:

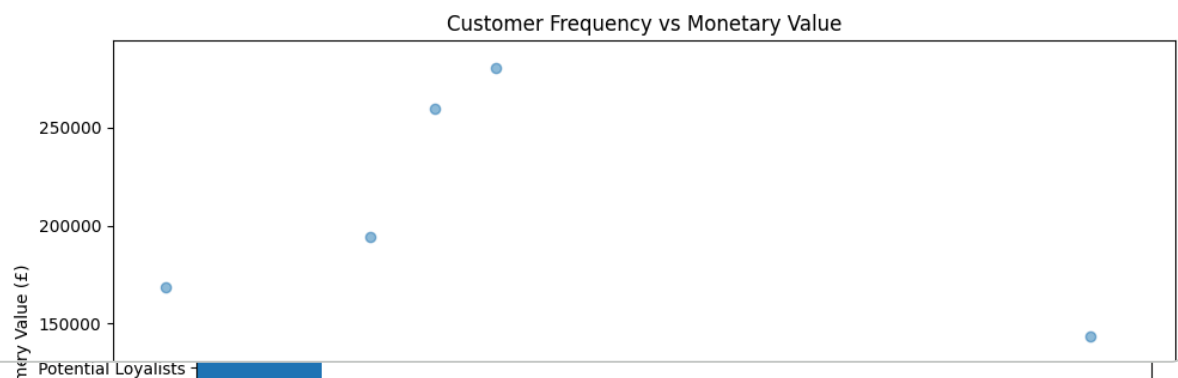
Customers Revenue Avg_Recency Avg_Frequency Avg_Monetary

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(
    rfm["Frequency"],
    rfm["Monetary"],
    alpha=0.5
)
```

```
plt.title("Customer Frequency vs Monetary Value")
plt.xlabel("Purchase Frequency")
plt.ylabel("Monetary Value (£)")
```

```
plt.tight_layout()
plt.show()
```



```
high_value_customers = (
    rfm
    .sort_values("Monetary", ascending=False)
    .head(20)
)
```

```
display(
    high_value_customers[
        [
            "CustomerID",
            "Recency",
            "Frequency",
            "Monetary",
            "RFM_Score",
            "Segment"
        ]
    ]
)
```

)]

	CustomerID	Recency	Frequency	Monetary	RFM_Score	Segment
1879	14911	1	201	143711.17	15	Champions
1689	14646	2	73	280206.02	15	Champions
4293	18198	10	69	259657.09	515	Champions
3728	17450	8	46	194390.79	15	Champions
3078	16684	4	28	66653.56	513	Potential Loyalists
1289	14096	4	17	65164.79	15	Champions
1879	14911	1	201	143711.17	15	Champions
996	13694	4	50	65039.62	15	Champions
55	12415	24	21	124914.53	14	Champions
1333	14156	10	55	117210.08	15	Champions
3771	17511	3	31	91062.38	15	Champions
2702	16029	39	63	80850.84	13	Loyal Customers
0	12346	326	1	77183.60	7	Lost Customers
3176	16684	4	28	66653.56	15	Champions
1289	14096	4	17	65164.79	15	Champions
996	13694	4	50	65039.62	15	Champions
2176	15311	1	91	60632.75	15	Champions
562	13089	3	97	58762.08	15	Champions
4093	17949	1	45	58510.48	15	Champions
2517	15769	7	26	56252.72	15	Champions
1983	15061	4	48	54534.14	15	Champions
1434	14298	8	44	51527.30	15	Champions
1284	14088	10	13	50491.81	15	Champions

```
total_customer_revenue = rfm["Monetary"].sum()

champion_revenue = (
    rfm.loc[
        rfm["Segment"] == "Champions",
        "Monetary"
    ].sum()
)

champion_customers = (
    (rfm["Segment"] == "Champions").sum()
)

total_customers = len(rfm)
```

```

print(
    f"Champions revenue share: "
    f"{champion_revenue / total_customer_revenue * 100:.2f}%"
)

print(
    f"Champions customer share: "
    f"{champion_customers / total_customers * 100:.2f}%"
)

```

```

Champions revenue share: 65.17%
Champions customer share: 22.06%

```

```

# =====
# PHASE 5 – STATISTICAL ANALYSIS
# =====

from scipy.stats import pearsonr, spearmanr

# Pearson correlation
pearson_corr, pearson_p = pearsonr(
    rfm["Frequency"],
    rfm["Monetary"]
)

# Spearman correlation
spearman_corr, spearman_p = spearmanr(
    rfm["Frequency"],
    rfm["Monetary"]
)

print("Pearson correlation:")
print(f"Correlation: {pearson_corr:.4f}")
print(f"P-value: {pearson_p:.6f}")

print("\nSpearman correlation:")
print(f"Correlation: {spearman_corr:.4f}")
print(f"P-value: {spearman_p:.6f}")

```

```

Pearson correlation:
Correlation: 0.5528
P-value: 0.000000

```

```

Spearman correlation:
Correlation: 0.8072
P-value: 0.000000

```

```

# Save cleaned transaction data
df.to_csv(
    "retail_clean.csv",
    index=False
)

# Save customer RFM data
rfm.to_csv(
    "customer_rfm.csv",
    index=False
)

print("Files created successfully!")

```

Files created successfully!

```
# Create ML dataset from customer-level RFM data

ml_df = rfm.copy()

# Create target variable
# 1 = customer is at risk / lost
# 0 = customer is active
ml_df["Churn"] = ml_df["Segment"].isin(
    ["At Risk", "Lost Customers"]
).astype(int)

# Features
features = ["Recency", "Frequency", "Monetary"]

X = ml_df[features]
y = ml_df["Churn"]

print("Dataset shape:", ml_df.shape)
print("\nTarget distribution:")
print(y.value_counts())

print("\nTarget percentage:")
print(y.value_counts(normalize=True) * 100)
```

Dataset shape: (4338, 10)

Target distribution:

Churn

0 2820

1 1518

Name: count, dtype: int64

Target percentage:

Churn

0 65.006916

1 34.993084

Name: proportion, dtype: float64

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

Training set: (3470, 3)

Testing set: (868, 3)

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
```

```

model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(random_state=42))
])

model.fit(X_train, y_train)

print("Model trained successfully!")

```

Model trained successfully!

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report
)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall   :", round(recall_score(y_test, y_pred), 4))
print("F1 Score :", round(f1_score(y_test, y_pred), 4))
print("ROC-AUC  :", round(roc_auc_score(y_test, y_prob), 4))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

```

Accuracy : 0.8802
Precision: 0.8846
Recall   : 0.7566
F1 Score : 0.8156
ROC-AUC  : 0.9686

```

```

Classification Report:

```

	precision	recall	f1-score	support
0	0.88	0.95	0.91	564
1	0.88	0.76	0.82	304
accuracy			0.88	868
macro avg	0.88	0.85	0.86	868
weighted avg	0.88	0.88	0.88	868

```

from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)
rf_prob = rf_model.predict_proba(X_test)[:, 1]

```

```

print("Random Forest Results")
print("-----")
print("Accuracy :", round(accuracy_score(y_test, rf_pred), 4))
print("Precision:", round(precision_score(y_test, rf_pred), 4))
print("Recall   :", round(recall_score(y_test, rf_pred), 4))
print("F1 Score :", round(f1_score(y_test, rf_pred), 4))
print("ROC-AUC  :", round(roc_auc_score(y_test, rf_prob), 4))

print("\nClassification Report:")
print(classification_report(y_test, rf_pred))

```

Random Forest Results

Accuracy : 0.9896

Precision: 0.9933

Recall : 0.977

F1 Score : 0.9851

ROC-AUC : 0.9995

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	564
1	0.99	0.98	0.99	304
accuracy			0.99	868
macro avg	0.99	0.99	0.99	868
weighted avg	0.99	0.99	0.99	868

```

# Make a copy of the customer transaction data
customer_data = df.dropna(subset=["CustomerID"]).copy()

customer_data["CustomerID"] = customer_data["CustomerID"].astype(int)

# Check date range
print("Minimum date:", customer_data["InvoiceDate"].min())
print("Maximum date:", customer_data["InvoiceDate"].max())

```

Minimum date: 2010-12-01 08:26:00

Maximum date: 2011-12-09 12:50:00

```

# Define a cutoff date
cutoff_date = pd.Timestamp("2011-10-01")

print("Cutoff date:", cutoff_date)

```

Cutoff date: 2011-10-01 00:00:00

```

# Historical transactions
historical = customer_data[
    customer_data["InvoiceDate"] < cutoff_date
].copy()

# Future transactions
future = customer_data[
    customer_data["InvoiceDate"] >= cutoff_date
].copy()

```

```
print("Historical transactions:", len(historical))
print("Future transactions:", len(future))
```

```
Historical transactions: 263705
Future transactions: 128987
```

```
customer_features = historical.groupby("CustomerID").agg(
    Recency=("InvoiceDate", lambda x:
        (cutoff_date - x.max()).days),

    Frequency=("InvoiceNo", "nunique"),

    Monetary=("Revenue", "sum"),

    AvgOrderValue=("Revenue", "mean"),

    ProductDiversity=("StockCode", "nunique")
).reset_index()

print(customer_features.head())
print("\nShape:", customer_features.shape)
```

	CustomerID	Recency	Frequency	Monetary	AvgOrderValue	ProductDive
0	12346	255	1	77183.60	77183.600000	
1	12347	59	5	2790.86	22.506935	
2	12348	5	4	1797.24	57.975484	
3	12350	240	1	334.40	19.670588	
4	12352	2	7	2194.31	31.347286	

```
Shape: (3616, 6)
```

```
future_purchase = (
    future.groupby("CustomerID")["InvoiceNo"]
    .nunique()
    .reset_index(name="FutureOrders")
)

customer_features = customer_features.merge(
    future_purchase,
    on="CustomerID",
    how="left"
)

customer_features["FutureOrders"] = (
    customer_features["FutureOrders"]
    .fillna(0)
)

customer_features["Repurchased"] = (
    customer_features["FutureOrders"] > 0
).astype(int)

print(customer_features["Repurchased"].value_counts())
```

```
Repurchased
1      1838
0      1778
Name: count, dtype: int64
```

```
print(
    customer_features["Repurchased"]
    .value_counts(normalize=True) * 100
)
```

```
Repurchased
1    50.829646
0    49.170354
Name: proportion, dtype: float64
```

```
from sklearn.model_selection import train_test_split

features = [
    "Recency",
    "Frequency",
    "Monetary",
    "AvgOrderValue",
    "ProductDiversity"
]

X = customer_features[features]
y = customer_features["Repurchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (2892, 5)
Testing set: (724, 5)
```

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logistic_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(random_state=42))
])

logistic_model.fit(X_train, y_train)

logistic_pred = logistic_model.predict(X_test)
logistic_prob = logistic_model.predict_proba(X_test)[: , 1]
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report
)

print("Logistic Regression")
```

```

print("-----")

print("Accuracy :", round(accuracy_score(y_test, logistic_pred), 4))
print("Precision:", round(precision_score(y_test, logistic_pred), 4))
print("Recall   :", round(recall_score(y_test, logistic_pred), 4))
print("F1 Score :", round(f1_score(y_test, logistic_pred), 4))
print("ROC-AUC  :", round(roc_auc_score(y_test, logistic_prob), 4))

print("\nClassification Report:")
print(classification_report(y_test, logistic_pred))

```

Logistic Regression

```

-----
Accuracy : 0.6878
Precision: 0.732
Recall   : 0.6087
F1 Score : 0.6647
ROC-AUC  : 0.7609

```

Classification Report:

	precision	recall	f1-score	support
0	0.66	0.77	0.71	356
1	0.73	0.61	0.66	368
accuracy			0.69	724
macro avg	0.69	0.69	0.69	724
weighted avg	0.69	0.69	0.69	724

```

from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    class_weight="balanced"
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)
rf_prob = rf_model.predict_proba(X_test)[: , 1]

```

```

print("Random Forest")
print("-----")

print("Accuracy :", round(accuracy_score(y_test, rf_pred), 4))
print("Precision:", round(precision_score(y_test, rf_pred), 4))
print("Recall   :", round(recall_score(y_test, rf_pred), 4))
print("F1 Score :", round(f1_score(y_test, rf_pred), 4))
print("ROC-AUC  :", round(roc_auc_score(y_test, rf_prob), 4))

print("\nClassification Report:")
print(classification_report(y_test, rf_pred))

```

Random Forest

```

-----
Accuracy : 0.6533
Precision: 0.663
Recall   : 0.6467
F1 Score : 0.6547

```

ROC-AUC : 0.7129

Classification Report:

	precision	recall	f1-score	support
0	0.64	0.66	0.65	356
1	0.66	0.65	0.65	368
accuracy			0.65	724
macro avg	0.65	0.65	0.65	724
weighted avg	0.65	0.65	0.65	724

```
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest"],
    "Accuracy": [
        accuracy_score(y_test, logistic_pred),
        accuracy_score(y_test, rf_pred)
    ],
    "Precision": [
        precision_score(y_test, logistic_pred),
        precision_score(y_test, rf_pred)
    ],
    "Recall": [
        recall_score(y_test, logistic_pred),
        recall_score(y_test, rf_pred)
    ],
    "F1 Score": [
        f1_score(y_test, logistic_pred),
        f1_score(y_test, rf_pred)
    ],
    "ROC-AUC": [
        roc_auc_score(y_test, logistic_prob),
        roc_auc_score(y_test, rf_prob)
    ]
})
```

```
comparison.round(4)
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Logistic Regression	0.6878	0.732	0.6087	0.6647	0.7609
1	Random Forest	0.6533	0.663	0.6467	0.6547	0.7129

```
feature_importance = pd.DataFrame({
    "Feature": features,
    "Importance": rf_model.feature_importances_
}).sort_values(
    "Importance",
    ascending=False
)
```

```
feature_importance
```

	Feature	Importance
2	Monetary	0.250532
0	Recency	0.215915
3	AvgOrderValue	0.210491
4	ProductDiversity	0.200872
1	Frequency	0.122190

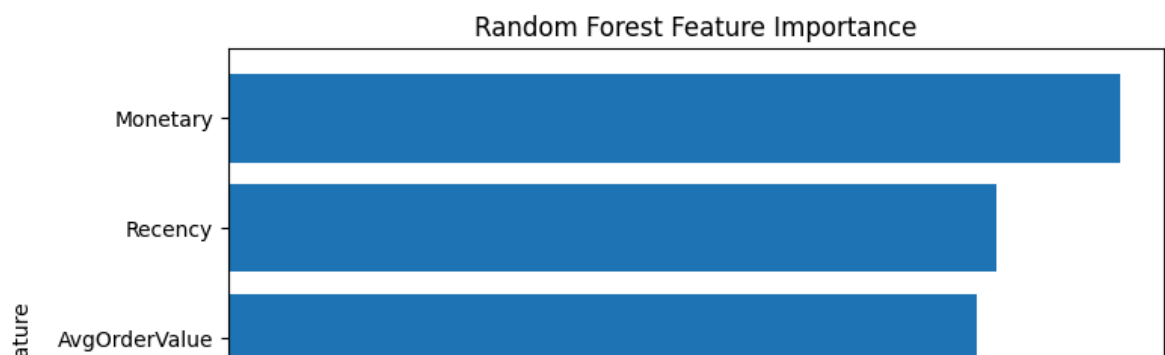
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.barh(
    feature_importance["Feature"],
    feature_importance["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Random Forest Feature Importance")
plt.gca().invert_yaxis()

plt.show()
```



```
# Generate repurchase probability for every customer

customer_features["Repurchase_Probability"] = logistic_model.predict_proba(
    customer_features[features]
)[: , 1]

customer_features["Predicted_Repurchase"] = (
    customer_features["Repurchase_Probability"] >= 0.5
).astype(int)

customer_features[
    [
        "CustomerID",
        "Recency",
        "Frequency",
        "Monetary",
        "AvgOrderValue",
        "ProductDiversity",
        "Repurchase_Probability",
        "Predicted_Repurchase"
    ]
]
```

```
]
].head(10)
```

	CustomerID	Recency	Frequency	Monetary	AvgOrderValue	ProductDiver
0	12346	255	1	77183.60	77183.600000	
1	12347	59	5	2790.86	22.506935	
2	12348	5	4	1797.24	57.975484	
3	12350	240	1	334.40	19.670588	
4	12352	2	7	2194.31	31.347286	
5	12353	134	1	89.00	22.250000	
6	12354	162	1	1079.40	18.610345	
7	12355	144	1	459.40	35.338462	
8	12356	175	2	2753.08	48.299649	
9	12358	80	1	484.86	40.405000	

```
def customer_priority(probability):
    if probability >= 0.75:
        return "High Probability"
    elif probability >= 0.50:
        return "Medium Probability"
    else:
        return "Low Probability"

customer_features["Repurchase_Priority"] = (
    customer_features["Repurchase_Probability"]
    .apply(customer_priority)
)

print(
    customer_features["Repurchase_Priority"]
    .value_counts()
)
```

```
Repurchase_Priority
Low Probability      2088
Medium Probability   914
High Probability     614
Name: count, dtype: int64
```

```
priority_customers = customer_features[
    customer_features["Repurchase_Priority"] == "Low Probability"
].sort_values(
    "Monetary",
    ascending=False
)

priority_customers[
    [
        "CustomerID",
        "Recency",
```

```

    "Frequency",
    "Monetary",
    "AvgOrderValue",
    "ProductDiversity",
    "Repurchase_Probability"
]
].head(20)

```

	CustomerID	Recency	Frequency	Monetary	AvgOrderValue	ProductDi
0	12346	255	1	77183.60	77183.600000	
2106	15749	165	3	44534.30	4453.430000	
1699	15098	112	3	39916.50	13305.500000	
1654	15032	186	3	4959.10	95.367308	
2575	16532	65	2	4516.80	1505.600000	
3595	18251	17	1	4314.72	269.670000	
3492	18087	28	2	4055.72	2027.860000	
273	12755	210	2	3811.95	952.987500	
508	13135	126	1	3096.00	3096.000000	
1836	15299	94	5	3035.01	607.002000	
272	12754	165	3	3010.74	47.789524	
1123	14145	86	3	2851.20	356.400000	
8	12356	175	2	2753.08	48.299649	
2274	16041	71	2	2739.80	547.960000	
183	12625	141	2	2683.82	31.207209	
1307	14439	249	1	2661.24	83.163750	
841	13680	267	2	2391.59	34.165571	
201	12653	78	2	2209.85	51.391860	
102	12501	266	1	2169.39	14.559664	
1546	14828	126	3	2139.76	125.868235	

```

# Create final customer scoring dataset

customer_scored = customer_features.copy()

# Add actual future behavior
customer_scored["Actual_Repurchased"] = customer_scored["Repurchased"]

# Calculate potential revenue at risk
customer_scored["Revenue_At_Risk"] = (
    customer_scored["Monetary"] *
    (1 - customer_scored["Repurchase_Probability"])
)

```

```
# Sort by revenue at risk
customer_scored = customer_scored.sort_values(
    "Revenue_At_Risk",
    ascending=False
)

customer_scored[
    [
        "CustomerID",
        "Recency",
        "Frequency",
        "Monetary",
        "Repurchase_Probability",
        "Actual_Repurchased",
        "Revenue_At_Risk"
    ]
].head(20)
```

	CustomerID	Recency	Frequency	Monetary	Repurchase_Probability
0	12346	255	1	77183.60	4.690366e-23
1699	15098	112	3	39916.50	3.619019e-04
2106	15749	165	3	44534.30	1.486159e-01
158	12590	141	2	9864.26	5.384765e-01
135	12557	88	4	9000.96	5.472028e-01
63	12435	9	2	7829.89	5.340875e-01
2575	16532	65	2	4516.80	2.179363e-01
3492	18087	28	2	4055.72	1.756134e-01
273	12755	210	2	3811.95	2.066760e-01
42	12409	8	3	11072.67	7.354028e-01
508	13135	126	1	3096.00	5.519718e-02
1654	15032	186	3	4959.10	4.429499e-01
2365	16182	2	4	6617.65	6.027445e-01
271	12753	65	5	16620.91	8.430949e-01
3595	18251	17	1	4314.72	4.024781e-01
382	12939	9	6	9036.12	7.206699e-01
435	13027	43	6	6912.00	6.524783e-01
314	12830	21	5	6414.32	6.400169e-01
270	12752	11	1	4366.78	5.089194e-01
1566	14866	10	5	6803.42	6.858469e-01

```
# Customers with low repurchase probability
low_probability = customer_scored[
    customer_scored["Repurchase_Probability"] < 0.5
]

print("Low probability customers:", len(low_probability))

print(
    "Historical revenue of low probability customers: £",
    round(low_probability["Monetary"].sum(), 2)
)

print(
    "Estimated revenue at risk: £",
    round(low_probability["Revenue_At_Risk"].sum(), 2)
)
```

```
Low probability customers: 2088
Historical revenue of low probability customers: £ 1061320.43
Estimated revenue at risk: £ 713926.92
```

```
probability_summary = (
    customer_scored
    .groupby("Repurchase_Priority")
    .agg(
        Customers=("CustomerID", "count"),
        Avg_Probability=("Repurchase_Probability", "mean"),
        Avg_Recency=("Recency", "mean"),
        Avg_Frequency=("Frequency", "mean"),
        Avg_Monetary=("Monetary", "mean"),
        Actual_Repurchase_Rate=("Actual_Repurchased", "mean")
    )
    .reset_index()
)

probability_summary["Actual_Repurchase_Rate"] *= 100

probability_summary.round(2)
```

	Repurchase_Priority	Customers	Avg_Probability	Avg_Recency	Avg_Fre
0	High Probability	614	0.89	24.19	
1	Low Probability	2088	0.36	132.06	
2	Medium Probability	914	0.61	45.63	

```
# Export datasets for Power BI

df.to_csv("retail_clean.csv", index=False)

rfm.to_csv("customer_rfm.csv", index=False)

customer_scored.to_csv("customer_predictions.csv", index=False)

print("Files exported successfully!")
```

```
Files exported successfully!
```

```
import os

for file in [
    "retail_clean.csv",
    "customer_rfm.csv",
    "customer_predictions.csv"
]:
    print(file, "→", round(os.path.getsize(file) / (1024 * 1024), 2), "MB")
```

```
retail_clean.csv → 48.01 MB
customer_rfm.csv → 0.18 MB
customer_predictions.csv → 0.36 MB
```

```
import os

print(os.listdir("/content"))
```

```
['.config', 'sample_data']
```

```
# =====
# RETAILIQ - COMPLETE DATA + RFM + ML PIPELINE
# =====

import pandas as pd
import numpy as np
import os
import shutil
import sqlite3

# ML
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report
)

# =====
# 1. LOAD DATA
# =====

excel_files = [
    f for f in os.listdir("/content")
    if f.lower().endswith((".xlsx", ".xls"))
]

print("Excel files found:", excel_files)

file_name = excel_files[0]

df = pd.read_excel(f"/content/{file_name}")

print("\nOriginal shape:", df.shape)
```

```

# =====
# 2. DATA CLEANING
# =====

df = df.drop_duplicates()

df = df.dropna(subset=["Description"])

df["InvoiceNo"] = df["InvoiceNo"].astype(str)

# Remove cancelled invoices
df = df[~df["InvoiceNo"].str.startswith("C")]

# Keep valid sales transactions
df = df[df["Quantity"] > 0]
df = df[df["UnitPrice"] > 0]

# Revenue
df["Revenue"] = df["Quantity"] * df["UnitPrice"]

# Date features
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

df["Year"] = df["InvoiceDate"].dt.year
df["Month"] = df["InvoiceDate"].dt.month
df["MonthName"] = df["InvoiceDate"].dt.month_name()

df["YearMonth"] = (
    df["InvoiceDate"]
    .dt.to_period("M")
    .astype(str)
)

df["DayOfWeek"] = df["InvoiceDate"].dt.day_name()
df["Hour"] = df["InvoiceDate"].dt.hour

print("\nCleaned shape:", df.shape)
print("Total Revenue: £", round(df["Revenue"].sum(), 2))
print("Orders:", df["InvoiceNo"].nunique())
print("Products:", df["StockCode"].nunique())
print("Customers:", df["CustomerID"].nunique())

# =====
# 3. RFM ANALYSIS
# =====

customer_df = df.dropna(subset=["CustomerID"]).copy()

customer_df["CustomerID"] = customer_df["CustomerID"].astype(int)

reference_date = (
    customer_df["InvoiceDate"].max()
    + pd.Timedelta(days=1)
)

rfm = customer_df.groupby("CustomerID").agg(
    Recency=(
        "InvoiceDate",
        lambda x: (reference_date - x.max()).days
    ),
    Frequency=(

```

```

        "InvoiceNo",
        "nunique"
    ),

    Monetary=(
        "Revenue",
        "sum"
    )
).reset_index()

# RFM scoring
rfm["R_Score"] = pd.qcut(
    rfm["Recency"],
    5,
    labels=[5, 4, 3, 2, 1]
)

rfm["F_Score"] = pd.qcut(
    rfm["Frequency"].rank(method="first"),
    5,
    labels=[1, 2, 3, 4, 5]
)

rfm["M_Score"] = pd.qcut(
    rfm["Monetary"].rank(method="first"),
    5,
    labels=[1, 2, 3, 4, 5]
)

rfm["RFM_Score"] = (
    rfm["R_Score"].astype(int)
    + rfm["F_Score"].astype(int)
    + rfm["M_Score"].astype(int)
)

def segment_customer(row):

    r = int(row["R_Score"])
    f = int(row["F_Score"])
    m = int(row["M_Score"])

    if r >= 4 and f >= 4 and m >= 4:
        return "Champions"

    elif r >= 3 and f >= 4:
        return "Loyal Customers"

    elif r >= 4 and f <= 2:
        return "New Customers"

    elif r <= 2 and f >= 3 and m >= 3:
        return "At Risk"

    elif r <= 2 and f <= 2:
        return "Lost Customers"

    else:
        return "Potential Loyalists"

rfm["Segment"] = rfm.apply(
    segment_customer,
    axis=1

```

```

)

print("\nRFM customers:", len(rfm))

print("\nRFM Segments:")
print(rfm["Segment"].value_counts())

# =====
# 4. TIME-BASED REPURCHASE DATASET
# =====

cutoff_date = pd.Timestamp("2011-10-01")

historical = customer_df[
    customer_df["InvoiceDate"] < cutoff_date
].copy()

future = customer_df[
    customer_df["InvoiceDate"] >= cutoff_date
].copy()

print("\nHistorical transactions:", len(historical))
print("Future transactions:", len(future))

# Historical customer features

customer_features = historical.groupby("CustomerID").agg(

    Recency=(
        "InvoiceDate",
        lambda x: (cutoff_date - x.max()).days
    ),

    Frequency=(
        "InvoiceNo",
        "nunique"
    ),

    Monetary=(
        "Revenue",
        "sum"
    ),

    AvgOrderValue=(
        "Revenue",
        "mean"
    ),

    ProductDiversity=(
        "StockCode",
        "nunique"
    )

).reset_index()

# Future purchase behavior

future_purchase = (
    future
    .groupby("CustomerID")["InvoiceNo"]
    .nunique()

```

```

        .reset_index(name="FutureOrders")
    )

    customer_features = customer_features.merge(
        future_purchase,
        on="CustomerID",
        how="left"
    )

    customer_features["FutureOrders"] = (
        customer_features["FutureOrders"]
        .fillna(0)
    )

    customer_features["Repurchased"] = (
        customer_features["FutureOrders"] > 0
    ).astype(int)

print("\nML dataset shape:", customer_features.shape)

print("\nRepurchase target:")
print(customer_features["Repurchased"].value_counts())

# =====
# 5. TRAIN / TEST SPLIT
# =====

features = [
    "Recency",
    "Frequency",
    "Monetary",
    "AvgOrderValue",
    "ProductDiversity"
]

X = customer_features[features]

y = customer_features["Repurchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# =====
# 6. LOGISTIC REGRESSION
# =====

logistic_model = Pipeline([
    ("scaler", StandardScaler()),

    ("classifier", LogisticRegression(
        random_state=42
    ))
])

logistic_model.fit(
    X_train,

```

```

        y_train
    )

    logistic_pred = logistic_model.predict(X_test)

    logistic_prob = (
        logistic_model
        .predict_proba(X_test)[: , 1]
    )

    print("\n=====")
    print("LOGISTIC REGRESSION")
    print("=====")

    print("Accuracy :",
          round(accuracy_score(y_test, logistic_pred), 4))

    print("Precision:",
          round(precision_score(y_test, logistic_pred), 4))

    print("Recall    :",
          round(recall_score(y_test, logistic_pred), 4))

    print("F1 Score :",
          round(f1_score(y_test, logistic_pred), 4))

    print("ROC-AUC   :",
          round(roc_auc_score(y_test, logistic_prob), 4))

    # =====
    # 7. RANDOM FOREST
    # =====

    rf_model = RandomForestClassifier(
        n_estimators=300,
        random_state=42,
        class_weight="balanced"
    )

    rf_model.fit(
        X_train,
        y_train
    )

    rf_pred = rf_model.predict(X_test)

    rf_prob = (
        rf_model
        .predict_proba(X_test)[: , 1]
    )

    print("\n=====")
    print("RANDOM FOREST")
    print("=====")

    print("Accuracy :",
          round(accuracy_score(y_test, rf_pred), 4))

    print("Precision:",
          round(precision_score(y_test, rf_pred), 4))

```

```

print("Recall   :",
      round(recall_score(y_test, rf_pred), 4))

print("F1 Score :",
      round(f1_score(y_test, rf_pred), 4))

print("ROC-AUC  :",
      round(roc_auc_score(y_test, rf_prob), 4))

# =====
# 8. RANDOM FOREST FEATURE IMPORTANCE
# =====

feature_importance = pd.DataFrame({
    "Feature": features,
    "Importance":
        rf_model.feature_importances_
}).sort_values(
    "Importance",
    ascending=False
)

print("\nFeature Importance:")
print(feature_importance)

# =====
# 9. CUSTOMER-LEVEL PREDICTIONS
# =====

customer_features["Repurchase_Probability"] = (
    logistic_model
    .predict_proba(
        customer_features[features]
    )[:, 1]
)

customer_features["Predicted_Repurchase"] = (
    customer_features["Repurchase_Probability"] >= 0.5
).astype(int)

def customer_priority(probability):
    if probability >= 0.75:
        return "High Probability"

    elif probability >= 0.50:
        return "Medium Probability"

    else:
        return "Low Probability"

customer_features["Repurchase_Priority"] = (
    customer_features["Repurchase_Probability"]
    .apply(customer_priority)
)

```

```
# Revenue at risk
customer_features["Revenue_At_Risk"] = (
    customer_features["Monetary"]
    *
    (1 - customer_features["Repurchase_Probability"])
)

# Actual result
customer_features["Actual_Repurchased"] = (
    customer_features["Repurchased"]
)

# =====
# 10. EXPORT CSV FILES
# =====

df.to_csv(
    "/content/retail_clean.csv",
    index=False
)

rfm.to_csv(
    "/content/customer_rfm.csv",
    index=False
)
```